

TEST BOOK

Progetto Cruise-IoT Monitor

Sistema IoT per il monitoraggio ambientale di cabine navali

Documento: Test Book v1.0

Progetto: Enterprise Cruise – IoT Platform (FASE 5)

Corso: GPOI – Gestione Progetti e Organizzazione d'Impresa

Team: Noemi Basaglia, Luigi Zara, Mattia Crepaldi, Lorenzo Malachin

Data esecuzione test: Mer 13/05/2026 (M5 – Consegna finale)

Versione software: v1.0

Test totali: 31 (PASS: 20 | N/D: 11 | FAIL: 0)

Stato: ✓ Tutti i test eseguibili superati

1. Registro delle revisioni

Rev.	Data	Autore	Descrizione
1.0	13/05/2026	Noemi Basaglia / Lorenzo Malachin	Prima emissione – esiti compilati

2. Scopo e campo di applicazione

Questo documento descrive i casi di test funzionali per il sistema Cruise-IoT Monitor. I test verificano il corretto funzionamento di ogni componente della pipeline IoT: dalla lettura del sensore DHT11 sul Raspberry Pi Pico WH, alla trasmissione TCP al gateway, alla pubblicazione MQTT su ThingsBoard, fino alla visualizzazione sulla dashboard.

I test contrassegnati N/D (Non Disponibile) riguardano scenari di errore che richiedono la modifica intenzionale della configurazione; non vengono eseguiti durante il collaudo finale per non compromettere il sistema funzionante.

Componente	Descrizione
wifidc.py	Connessione WiFi del Raspberry Pi Pico WH (T-01 ÷ T-05)
misurazione.py + dc.py	Lettura sensore DHT11 e trasmissione TCP al gateway (T-06 ÷ T-10)
dc.py → iotgwda.py	Canale TCP client-gateway; gestione multi-threading (T-11 ÷ T-15)
iotgwda.py	Gateway IoT: ricezione, aggregazione, pubblicazione MQTT (T-16 ÷ T-22)
cripto.py	Cifratura/decifratura Base64 del payload (T-23 ÷ T-25)
Dashboard ThingsBoard	Visualizzazione real-time, allarmi, mobile, multi-cabina (T-26 ÷ T-31)

3. Ambiente di test

Hardware

- Raspberry Pi Pico WH – esecuzione dc.py, misurazione.py, wifidc.py, cripto.py
- Sensore DHT11 – collegato al GPIO 0 (alimentazione 3.3V)
- PC / Notebook – esecuzione iotgwda.py e istanza ThingsBoard via Docker
- Smartphone / Tablet – verifica accessibilità dashboard mobile
- Rete WiFi scolastica (SSID: Vodafone-IoT)

Software e parametri

MicroPython: ≥1.22 su Raspberry Pi Pico WH

Python: 3.10+ su server (libreria paho-mqtt)

ThingsBoard: Community Edition via Docker – localhost:8080; broker MQTT porta 1883

TOPIC MQTT: v1/devices/me/telemetry

Porta TCP server: 6767 (fallback automatico 6769)

Intervallo lettura DC: 2 secondi

Finestra aggregazione gateway: 5 secondi (TEMPO_RILEVAZIONE)

Dispositivi testati: DC-001 (cab.1 ponte 1), DC-002 (cab.2 ponte 1), DC-003 (cab.3 ponte 2)

4. Legenda esiti

Esito	Significato
PASS	Il comportamento osservato corrisponde al risultato atteso
N/D	Test non eseguito: scenario di errore che richiederebbe modifica intenzionale del sistema funzionante

5. Riepilogo esecuzione

Totale	PASS	FAIL	N/D	Data esecuzione	Versione	Tester	Firma
31	20	0	11	13/05/2026	v1.0	Tutto il team	

6. Casi di test

MODULO 1 – Connessione WiFi (wifidc.py)					
ID	Descrizione	Precondizioni	Procedura di test	Risultato atteso	Esito
T-01	Caricamento credenziali WiFi da file	File wifipico.json presente sul Pico con campi 'ssid' e 'pw'	1. Eseguire wifidc.py2. Osservare console	SSID e password letti correttamente; nessun errore stampato	PASS
T-02	File wifipico.json mancante – gestione errore	File wifipico.json assente	1. Rimuovere wifipico.json2. Eseguire wifidc.py	Messaggio 'Errore: File wifipico.json non trovato!'	N/D
T-03	Connessione WiFi con credenziali corrette	Rete 'Vodafone-IoT' disponibile; wifipico.json valido	1. Eseguire wifidc.py2. Attendere handshake	Console mostra IP, Subnet, Gateway; stato wlan=3 (Link Up)	PASS
T-04	Connessione WiFi con password errata	Rete disponibile; password sbagliata	1. Impostare password errata2. Eseguire wifidc.py	Stato -3 (Link BadAuth); LED lampeggia 3 volte	N/D
T-05	Disabilitazione powersaving	Pico avviato	1. Chiamare set_powersaving(wlan, disabilita=True)	Console mostra 'Powersaving: Disabilitato'	PASS

MODULO 2 – Lettura sensore DHT11 (misurazione.py + dc.py)					
ID	Descrizione	Precondizioni	Procedura di test	Risultato atteso	Esito
T-06	Lettura valida da sensore DHT11	Sensore DHT11 collegato al GPIO 0; alimentazione 3.3V	1. Avviare dc.py2. Attendere primo ciclo (2s)	Temperatura 0–50°C; umidità 0–100%; nessuna eccezione	PASS
T-07	Sensore non collegato – gestione eccezione	Sensore scollegato	1. Avviare dc.py senza sensore	OSError catturata; messaggio '[DCxxx] ERRORE'; socket chiuso	N/D
T-08	Arrotondamento a 2 decimali	Sensore operativo	1. Leggere 10 misurazioni2. Verificare campo 'temperature' nel JSON	Ogni valore float ha al massimo 2 cifre decimali	PASS
T-09	Scelta identità valida da da.json	da.json con chiavi DC-001/002/003	1. Avviare dc.py2. Inserire 'DC-001'	Parametri letti: IP=192.168.1.72, porta=6767, cabina=1, ponte=1	PASS
T-10	Identità non presente in da.json	da.json presente	1. Avviare dc.py2. Inserire 'DC-999'	Messaggio 'ERRORE: DC-999 non trovata'; lista identità stampata; SystemExit	N/D

MODULO 3 – Trasmissione TCP DC → Gateway (dc.py → iotgwda.py)					
ID	Descrizione	Precondizioni	Procedura di test	Risultato atteso	Esito

T-11	Connessione TCP al Gateway riuscita	iotgwda.py in esecuzione su 0.0.0.0:6767	1. Avviare iotgwda.py2. Avviare dc.py con DC-001	Gateway stampa 'Nuovo client connesso'; DC stampa 'Connesso al Gateway IoT'	PASS
T-12	Gateway non raggiungibile – gestione errore DC	iotgwda.py non in esecuzione	1. Avviare dc.py senza gateway attivo	Eccezione catturata; 'ERRORE connessione al gateway'; SystemExit	N/D
T-13	Formato JSON corretto del messaggio DC	Pipeline DC→Gateway operativa	1. Avviare pipeline2. Leggere print console DC	JSON contiene: identita, cabina, ponte, temperature, humidity	PASS
T-14	Invio continuo ogni 2 secondi	Pipeline operativa	1. Avviare pipeline2. Contare messaggi per 10 secondi	Almeno 4 messaggi ricevuti in 10 secondi	PASS
T-15	Connessioni multiple simultanee (multi-thread)	iotgwda.py avviato; DC-001 e DC-002 disponibili	1. Avviare iotgwda.py2. Avviare DC-001 e DC-002	Gateway gestisce entrambi in thread separati senza interferenze	PASS

MODULO 4 – Gateway IoT e pubblicazione MQTT (iotgwda.py)					
ID	Descrizione	Precondizioni	Procedura di test	Risultato atteso	Esito
T-16	Caricamento parametri.json corretto	File Configurazione/p arametri.json presente	1. Avviare iotgwda.py2. Osservare console	Porta 6767 in ascolto; 'Gateway IoT in attesa di connessioni'	PASS
T-17	File parametri.json mancante	File rimosso	1. Avviare iotgwda.py senza il file	Messaggio 'Errore lettura parametri'; sys.exit()	N/D
T-18	Token MQTT corretto per DC-001	tokens.json con token DC-001; ThingsBoard su 127.0.0.1:1883	1. Avviare pipeline completa2. Verificare log al primo dato DC-001	Gateway stampa 'MQTT connesso per dispositivo DC-001'	PASS
T-19	Identità senza token – gestione errore	DC invia identità non in tokens.json	1. Usare identità non in tokens.json	Messaggio 'ERRORE: nessun token trovato'; thread terminato	N/D
T-20	Calcolo media su finestra 5 secondi	Pipeline operativa; TEMPO_RILEVAZIONE=5	1. Avviare pipeline2. Attendere 5 secondi3. Leggere console gateway	Gateway stampa media calcolata su ~2-3 letture; valore in range 0-50°C	PASS
T-21	Formato JSON pubblicato su MQTT	Pipeline operativa; broker MQTT attivo	1. Sottoscrivere topic 'v1/devices/me/telemetry'2. Leggere messaggio ricevuto	JSON contiene: cabina, ponte, temperature, humidity, dataeora, invionumero,	PASS

				identita	
T-22	Porta occupata – cambio automatico	Porta 6767 già in uso	1. Occupare porta 67672. Avviare iotgwda.py	Messaggio 'Cambio porta automaticament e'; gateway avviato su 6769	N/D

MODULO 5 – Cifratura Base64 (cripto.py)					
ID	Descrizione	Precondizioni	Procedura di test	Risultato atteso	Esito
T-23	Cifratura stringa semplice	Modulo cripto.py importato	1. Chiamare criptazione('hello')2. Verificare output	Output: 'aGVsbG8=' (Base64 di 'hello'); tipo str	PASS
T-24	Decifratura – round-trip corretto	Modulo cripto.py importato	1. Chiamare decriptazione(criptazione('test123'))	Output uguale all'originale: 'test123'	PASS
T-25	Cifratura JSON completo	Modulo cripto.py importato	1. Cifrare json.dumps(dato_iot)2. Decifrare e confrontare	JSON originale recuperabile dopo round-trip; nessuna perdita	PASS

MODULO 6 – Dashboard ThingsBoard					
ID	Descrizione	Precondizioni	Procedura di test	Risultato atteso	Esito
T-26	Dati visibili entro 5 secondi	Pipeline completa attiva; dashboard aperta nel browser	1. Avviare pipeline2. Avviare cronometro3. Osservare gauge	Valori aggiornati entro 5 secondi dall'acquisizione del sensore	PASS
T-27	Widget gauge temperatura – range corretto	Pipeline operativa	1. Osservare gauge2. Confrontare con console	Gauge mostra stesso valore del JSON MQTT (± 0.01); range 0-50°C	PASS
T-28	Grafico time-series ultime 24h	Pipeline attiva da almeno 10 minuti	1. Aprire dashboard2. Verificare widget time-series	Storico con timestamp corretti; asse X copre le ultime 24h	PASS
T-29	Allarme al superamento soglia temperatura	Soglia configurata su ThingsBoard; pipeline attiva	1. Riscaldare il sensore2. Far superare la soglia3. Osservare dashboard	Allarme attivato automaticament e entro il prossimo ciclo ($\leq 5s$)	PASS
T-30	Accessibilità da dispositivo mobile	Smartphone sulla stessa rete scolastica	1. Aprire browser su smartphone2. Navigare all'IP ThingsBoard:porta	Dashboard caricata; widget leggibili senza zoom	PASS
T-31	Gestione multi-cabina – Entity Alias	DC-001 e DC-002 configurati; pipeline multi-device attiva	1. Avviare DC-001 e DC-0022. Verificare Entity Alias sulla dashboard	Dati cabina 1/ponte 1 e cabina 2/ponte 1 mostrati separatamente	PASS

I test N/D corrispondono a scenari di guasto/errore non eseguiti durante il collaudo finale. Sono inclusi nel Test Book a documentazione della copertura di test prevista.